

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Interpreting Algorithmic Problems

by **Neel Nanda** 31st Dec 2022

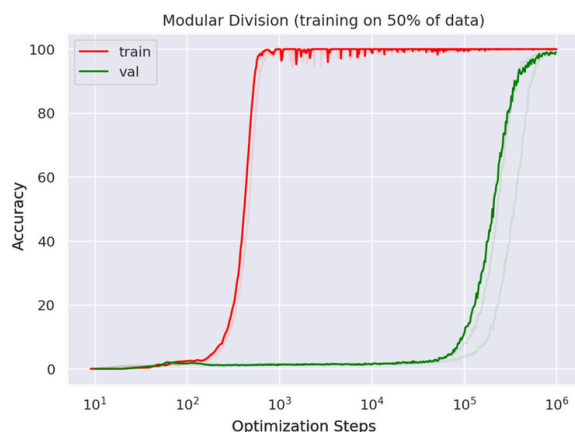
This is the fourth post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#)[°], then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Motivation

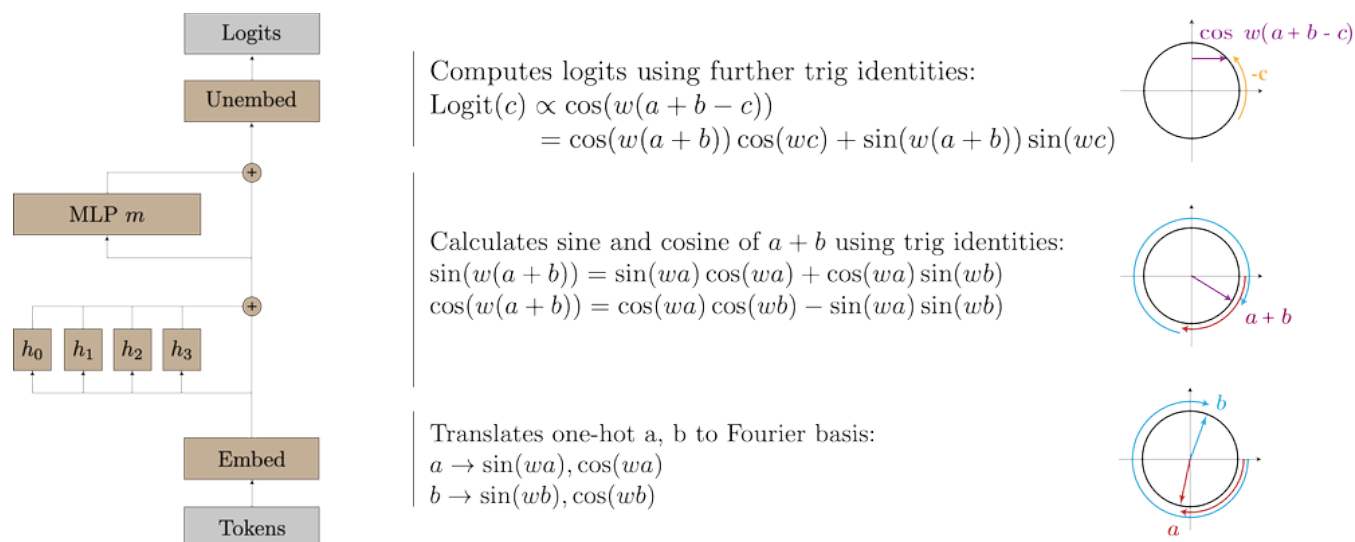
Motivating paper: [A Mechanistic Interpretability Analysis of Grokking](#)[°]

When models are trained on synthetic, algorithmic tasks, they often learn to do some clean, interpretable computation inside. Choosing a suitable task and trying to reverse engineer a model can be a rich area of interesting circuits to interpret! In some sense, this is interpretability on easy mode - the model is normally trained on a single task (unlike language models, which need to learn everything about language!), we know the exact ground truth about the data and optimal solution, and the models are tiny. So why care?

I consider [my work on grokking](#)[°] to be an interesting case study of this work going well. Grokking (shown below) is a mysterious phenomena where, when small models are trained on algorithmic tasks (eg modular addition or modular division), they initially memorise the training data. But when they *keep* being trained on that data for a really long time, the model suddenly(ish) figures out how to generalise!



In my work, I simplified their setup even further, by training a 1 Layer transformer (with no LayerNorm or biases) to do modular addition and reverse engineered the weights to understand what was going on. And it turned out to be doing a funky trig-based algorithm (shown below), where the numbers are converted to frequencies with a memorised Discrete Fourier Transform, added using trig identities, and converted back to the answer! Using this, we looked inside the model and identified that despite seeming to have plateaued, in the period between memorising and “grokking”, the model is actually slowly forming the circuit that does generalise. But so long as the model still has the memorising circuit, this adds too much noise to have good test loss. Grokking occurs when the generalising circuit is so strong that the model decides to “clean-up” the memorising circuit, and “uncovers” the mature generalising circuit beneath, and suddenly gets good test performance.



OK, so I just took this as an excuse to explain my paper to you. Why should you care? I think that the general lesson from this, that I’m excited to see applied elsewhere, is using toy algorithmic models to analyse a phenomena we’re confused about. Concretely, given a confusing phenomena like grokking, I’d advocate the following strategy:

1. **Simplify** to the minimal setting that exhibits the phenomena, yet is complex enough to be interesting
2. **Reverse-engineer** the resulting model, in as much detail as you can
3. **Extrapolate** the insights you've learned from the reverse-engineered model - what are the broad insights you've learned? What do you expect to generalise? Can you form any automated tests to detect the circuits you've found, or any of their motifs?
4. **Verify** by looking at other examples of the phenomena and seeing whether these insights actually hold (larger models, different tasks, even just earlier checkpoints of the model or different random seeds)

Grokking is an example in a science of deep learning context - trying to uncover mysteries about how models learn and behave. But this same philosophy also applies to understanding confusing phenomena in language models, and building toy algorithmic problems to study those!

Anthropic's [Toy Models of Superposition](#) is an excellent example of this done well, for the case of [superposition in linear bottleneck dimensions](#) of a model (like value vectors in attention heads, or the residual stream). Concretely, they isolated out the key traits as being where high dimensional spaces are projected to a low dimensional space, and then mapped back to a high dimensional space, with no non-linearity in between. And got extremely interesting and rich results! (More on these in the next post)

More broadly, because algorithmic tasks are often cleaner and easier to interpret, and there's a known ground truth, it can be a great place to practice interpretability! Both as a beginner to the field trying to build intuitions and learn techniques, and to refine our understanding of the *right* tools and techniques. It's much easier to validate a claimed approach to validate explanations (like [causal scrubbing](#)) if you can run it on a problem with an understood ground truth!

Overall, I'm less excited about algorithmic problem interpretability in general than some of the other categories of open problems, but I think it can be a great place to start and practice. I think that building a toy algorithmic model for a confusing phenomena is hard, but can be really exciting if done well!

Resources

- **Demo:** Reverse engineering how a small transformer can re-derive positional embeddings ([colab](#)), with [an accompanying video walkthrough](#)

- **Demo:** My grokking work reverse-engineering modular addition - [write-up](#)° and [colab](#)
 - Note - I'd guess that none of the Fourier stuff to generalise to other problems, but some insights and the underlying mindsets will. Modular addition is weird!

Tips

- Approach the reverse engineering scientifically. Go in and try to form hypotheses about what the model is doing, and how it might have solved the task. Then go and *test* these hypotheses, and try to look for evidence for them, and evidence to falsify them. Use these hypotheses to help you prioritise and focus among all of the possible directions you can go in - accept that they'll probably be wrong in some important ways (I did *not* expect modular addition to be using Fourier Transforms!), but use them as a guide to figure out *how* they're wrong.
 - Concretely, try to think about how you would implement a solution if you were designing the model weights. This is pretty different from writing code! Models tend to be very good at parallelised, vectorised solutions that heavily rely on linear algebra, with limited use of non-linear activation functions.
 - A useful way to understand how to think like a transformer is to look at [this paper](#), which introduces a programming language called RASP that aims to mimic the computational model of the transformer.
- These problems often involve training your own model, rather than just analysing a model someone else trained. These are such toy problems that training is pretty easy, but it can still be a headache! I recommend cribbing training code from somewhere (eg the demos above).
- Subtle details of how you set up the problem can make life much easier or harder for the model. Think carefully about things and try many variations.
 - Eg what is the loss function, whether there are special tokens in the context (eg to mark the end of the first input and the start of the second input or the output) or if the model needs to infer these for itself, etc.
- Rule of thumb: Small models are easier to interpret (it matters more to have few layers than narrow layers). Before analysing a model, check that the next smallest model can't also do the task!
 - Eg, decrease the number of transformer layers, remove LayerNorm, try attention-only, try an MLP (the classic neural network, not a transformer) with one hidden layer (or two).

- When building a toy model to capture something about a real model, you want to be especially thoughtful about how you do this! Try to distill out exactly what traits of the model determine the property that you care about. You want to straddle the fine line between too simple to be interesting and too complex to be tractable.
 - This is a particularly helpful thing to get other people's feedback on, and to spend time red-teaming

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- Good beginner problems:
 - A 3.1 - Sorting fixed-length lists. (format - `START 4 6 2 9 MID 2 4 6 9`)
 - How does difficulty change with the length of the list?
 - **A* 3.2** - Sorting variable-length lists.
 - What's the sorting algorithm? What's the longest list you can get to? How is accuracy affected by longer lists?
 - A 3.3 - Interpret a 2L MLP (one hidden layer) trained to do modular addition (very analogous to my grokking work)
 - A 3.4 - Interpret a 1L transformer trained to do modular subtraction (very analogous to my grokking work)
 - A 3.5 - Taking the minimum or maximum of two ints
 - A 3.6 - Permuting lists
 - A 3.7 - Calculating sequences with a Fibonacci-style recurrence relation.
 - (I.e. predicting the next element from the previous two)
- Some harder concrete algorithmic problems to try interpreting
 - **B* 3.8** - 5 digit addition/subtraction
 - What if you reverse the order of the digits? (e.g. inputting and outputting the units digit first). This order might make it easier to compute the digits since e.g. the tens digit depends on the units digit.
 - 5 digits is a good choice because we have prior knowledge that [grokking happens there, and it seems likely related to the modular addition algorithm](#). You might need to play around with training setups and hyperparameters if you varied the number of digits.

- **B* 3.9** - Predicting the output to simple code functions. E.g. predicting the bold text in problems like


```
a = 1 2 3
a[2] = 4
a -> 1 2 4
```
- **B* 3.10** - Graph theory problems like [this](#)
 - Not sure of the right input format - try a bunch! Check this out: <https://jacobbrazeal.wordpress.com/2022/09/23/gpt-3-can-find-paths-up-to-7-nodes-long-in-random-graphs/#comment-248>
- **B* 3.11** - Train a model on multiple algorithmic tasks that we understand (eg train a model to do modular addition *and* modular subtraction, by learning two different outputs). Compare this to a model trained on each task.
 - What happens? Does it learn the same circuits? Is there superposition?
 - If doing grokking-y tasks, how does that interact?
- **B* 3.12** - Train models for [automata](#) tasks and interpret them - do your results match the theory?
- **B* 3.13** - In-Context Linear Regression, as described in [Garg et al](#) - the transformer is given a sequence $(x_1, y_1, x_2, y_2, \dots)$, where $y_i = Ax_i + b$ and A and b are different for each prompt and need to be learned in-context ([see code here](#))
 - Tip - train a 2L or 3L model on this and give it width 256 or 512, this should be much easier to interpret than their 12L one.
 - **C* 3.14** - The other problems in the paper that are in-context learned - sparse linear functions, 2L networks & decision trees
- C 3.15 - 5 digit (or binary) multiplication
- B 3.16 - Predict repeated subsequences in randomly generated tokens, and see if you can find and reverse engineer [induction heads](#) (see [my grokking paper](#) for details.)
 - Tip: Using shortformer style positional embeddings (see [the TransformerLens docs](#)), this seems to make things cleaner
- B-C 3.17 - Choose your own adventure: Find your own algorithmic problem!
 - [Leetcode easy](#) is probably a good source of problems.
- **B* 3.18** - Build a toy model of [Indirect Object Identification](#) - train a tiny attention only model on an algorithmic task simulating Indirect Object Identification, and reverse-engineer the learned solution. Compare this to [the circuit found in GPT-2 Small](#) - is it

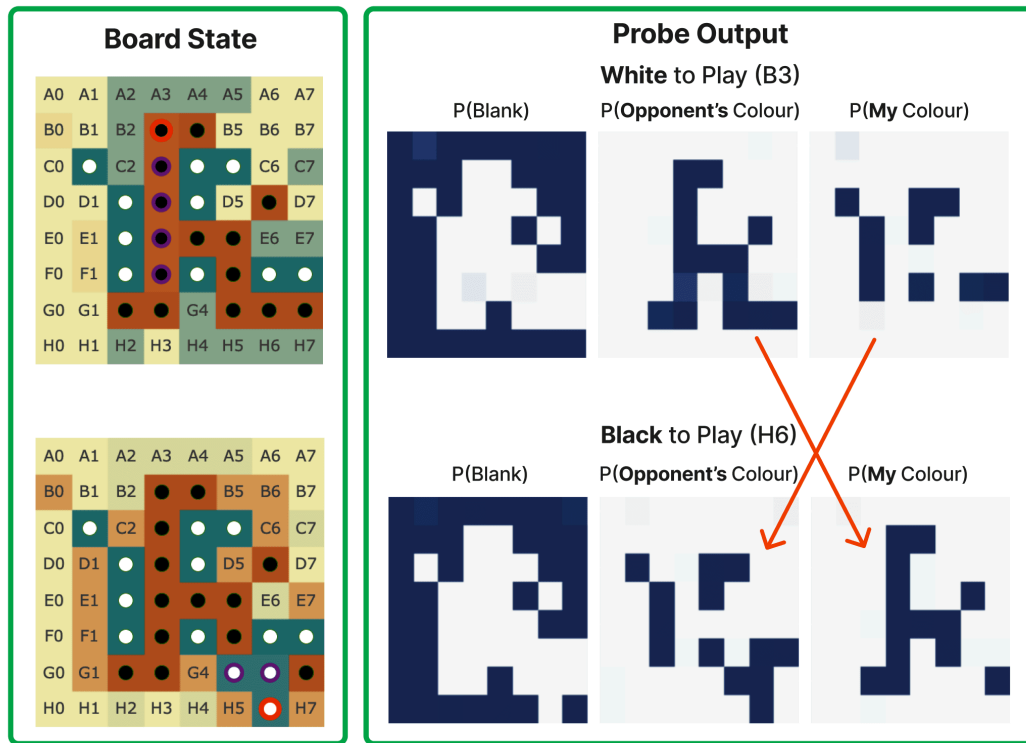
the same algorithm? We often build toy models to model things in real models, it'd be interesting to validate this by going the other way

- Context: IOI is the grammatical task of figuring out that sentences like "When John and Mary went to the store, John gave a bottle of milk to" maps to "Mary",. Interpretability in the Wild was a paper reverse engineering the 25 head circuit behind this in GPT-2 Small.
- The MVP would be just giving the model a BOS, a MID and the names, eg `BOS John Mary John MID -> Mary`. To avoid trivial solutions, make eg half the data 3 distinct names like `John Mary Peter` which maps to a randomly selected one of those names. I'd try training a 3L attn-only model to do this.
- **C* 3.19** - There's a bunch of follow-up questions: Is this consistent across random seeds, or can other algorithms be learned? Can a 2L model learn this? What happens if you add MLPs or more layers?
- **C* 3.20** - Reverse-engineer [Othello-GPT \(summary here\)](#) - the paper trains a GPT-style model to predict the next move in randomly generated Othello games (ie, it predicts legal moves, but doesn't learn strategy). Can you reverse-engineer the algorithms the model learns?
 - In particular, they train probes and use this to recover the model's internal model of the board state, and show that they can intervene to change what the model thinks, where it makes legal moves in the edited board. Can you reverse-engineer the features the probes find?
 - Tip: They train an 8L model, I expect you can get away with way smaller. They also use one hidden layer MLP probes, I'd start with using a linear probe for simplicity.
- Exploring questions about language models:
 - **A* 3.21** - Train a one layer attention-only transformer with rotary to predict the previous token, and reverse engineer how it does this.
 - An even easier way to do this is by training just the QK-circuit in an attention head and the embedding matrix to maximise the attention paid to the previous token (with mean-squared error loss) - this will cut out other crap the model might be doing.
 - **B* 3.22** - Train a three layer attention-only transformer to perform the [Indirect Object Identification](#) task (and *just* that task! You can algorithmically generate training data). Can it do the task? And if so, does it learn [the same circuit that was found in GPT-2 Small](#)?
 - **B* 3.23** - Re-doing my modular addition analysis with GELU. How does this change things, if at all?

- Bonus: Doing it for any algorithmic task! This probably works best on those with sensible and cleanly interpretable neurons
- **C* 3.24** - How does memorization work?
 - Idea 1: Train a one hidden layer MLP to memorise random data.
 - Possible setup: There are two inputs, each in $\{0, 1, \dots, n-1\}$. These are one-hot encoded, and stacked (so the input has dimension $2n$). Each pair has a randomly chosen output label, and the training set consists of all possible pairs.
 - Idea 2: Try training a transformer on a fixed set of random strings of tokens?
- **B-C* 3.25** - Comparing different [dimensionality reduction](#) techniques (e.g. [PCA/SVD](#), [t-SNE](#), [UMAP](#), [NMF](#), [the grand tour](#)) on modular addition (or any other problem that you feel you understand!)
 - These techniques have been used on [AlphaZero](#), [Understanding RL Vision](#), and [The Building Blocks of Interpretability](#) - which should be a good starting point for how.
 - B 3.26 - In modular addition, look at what these do on different weight matrices - can you identify which weights matter most? Or which neurons form clusters for each frequency? Can you find anything if you look at activations?
- **C* 3.27** - Is [direct logit attribution](#) always useful? Can you find examples where it's highly misleading?
 - I'd focus on problems where a component's output is mostly intended to suppress incorrect logits (so it improves the correct log prob but *not* the correct logit)
- Exploring broader deep learning mysteries - can you build a toy, algorithmic model to better understand these?
 - **D* 3.28** - The [Lottery Ticket Hypothesis](#)
 - **D* 3.29** - [Deep Double Descent](#)
- Othello-GPT is a model trained to play random legal moves in Othello, and learns a linear emergent world model, modelling the board state (despite only ever seeing and predicting moves!) I did some [work on it](#) and [outline directions of future work](#). I think it sits at a good intersection of complex enough to be interesting (much more complex than any algorithmic model that's been well understood!) yet simple enough to be tractable!

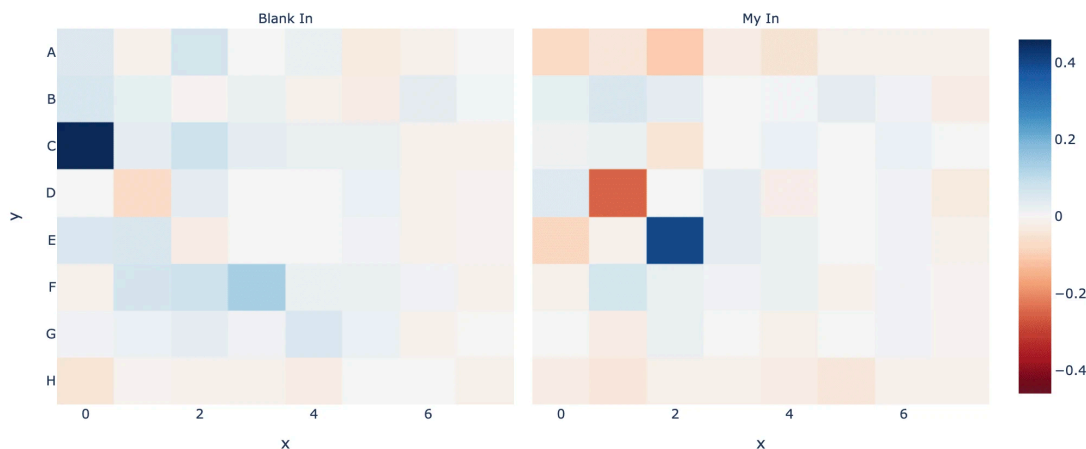
Othello-GPT's Linear Model of Board State

Input: F4 F3 D2 F5 G2 F2 G3 C4 E5 F6 D6 E2 B4 C5 G7 C1 G6 F7 G5 C3 **B3 H6**



-
- I go into a *lot* more detail in the post including on concrete problems and [give code](#) to build off of, so I think this can be a great place to start. The following is a brief summary.
- **A*** 3.30 Trying one of [the concrete starter projects](#)°
- **B-C*** 3.31 [Looking for modular circuits](#)° - try to find the circuits used to compute the world model and to use the world model to compute the next move, and understand each in isolation and use this to understand how they fit together. See what you can learn about finding modular circuits in general.
 - Example: Interpreting a neuron in the middle of the model by multiplying its input weights by the probe directions, to see what board state it looks at

Input weights in terms of the probe for neuron L5N1393



- **B-C*** 3.32 [Neuron Interpretability and Studying Superposition](#)° - try to understand the model's MLP neurons, and explore what techniques do and do not work. Try to build our understanding of how to understand transformer MLPs in general.
 - We're really bad at MLPs in general, but certain things are much easier here - eg it's easy to automatically check whether a neuron only activates when a condition is met, because all features here are clean and algorithmic
- **B-C*** 3.33 [A Transformer Circuit Laboratory](#)° - Explore and test other conjectures about transformer circuits, eg can we figure out how the model manages memory in the residual stream?

Previous:

200 COP in MI: Looking for Circuits in the Wild

3 comments 16 karma

Next:

200 COP in MI: Exploring Polysemanticity and Superposition

1 comments 34 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 14 Mechanistic Interpretability Quickstart Guide
- 14 Mech Interp Challenge: August - Deciphering the First Unique Character Model
- 11 200 COP in MI: Analysing Training Dynamics

[Moderation Log](#)

More from Neel Nanda

19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0
32	Models May Behave Worse When Eval Aware...	Senthooran Rajamanoharan, Neel ...	2d	0
65	models have some pretty funny attractor sta...	aryaj, Senthooran Rajamanoharan, ...	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike's-Eye View of ARC's Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0